

Assignment No: 5

Student Details

Name: Tanushka Patil

PRN / Roll No: 123B1B036

Class: BTech

Division: A

Subject: DevOps

Title

Multi-Container Application using Docker Compose (Flask Web App + Redis)

Aim

To explore and create a multi-container application consisting of a simple web application and a database service, and configure services, ports, environment variables, volumes and networking using a compose.yaml file.

Theory

Containers and Multi-Container Applications

A container packages an application together with its libraries and dependencies so it runs the same way on any machine. Real applications usually consist of several components – a web front end, an API, a database, a cache – and best practice is to run each component in its own container. Such a set of cooperating containers is called a multi-container application.

Docker Compose

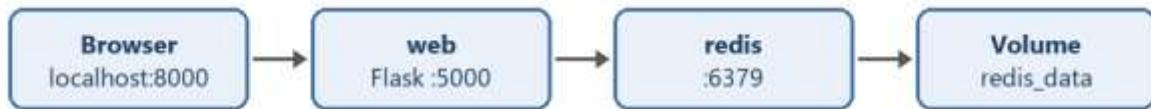
Docker Compose is a tool for defining and running multi-container applications. All services are described in a single YAML file (compose.yaml), and the whole stack is started or stopped with one command.

- **services:** each container in the application (image or build context, command, restart policy).
- **ports:** maps a host port to a container port (HOST:CONTAINER), e.g. "8000:5000".
- **environment:** passes configuration values such as hostnames, credentials or app settings to a container.
- **volumes:** persistent storage that survives container removal; named volumes are managed by Docker.
- **networks:** user-defined networks; containers on the same network reach each other by service name through Docker's built-in DNS.
- **depends_on:** controls start-up order between services.
- **build:** builds an image from a Dockerfile instead of pulling one.

Application Architecture

The application has two services on a shared bridge network (app_net): a Python Flask web service (flask_web) that listens on port 5000 inside the container and is published on host port 8000, and a Redis database service (redis_db) that stores a visit counter. Redis data is persisted in the named volume redis_data, so the count survives restarts.

Figure: Multi-container architecture (network: app_net)



Code / Implementation

Step 1: Prerequisites

```
docker --version
docker compose version
```

Step 2: Project structure

```
Assignment5_MultiContainer/
├── compose.yaml
├── web/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
```

web/app.py – Flask web application

```
import os
from flask import Flask
import redis

app = Flask(__name__)
cache = redis.Redis(host=os.environ.get("REDIS_HOST", "redis"),
                    port=int(os.environ.get("REDIS_PORT", 6379)))

@app.route("/")
def index():
    visits = cache.incr("visits")
    name = os.environ.get("APP_NAME", "Multi-Container App")
    return f"<h1>{name}</h1><p>This page has been visited {visits} times.</p>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

web/requirements.txt

```
flask==3.0.3
redis==5.0.8
```

web/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

compose.yaml – Services, ports, environment, volumes and networks

```
services:
  web:
    build: ./web
    container_name: flask_web
    ports:
      - "8000:5000"
    environment:
      - APP_NAME=DevOps Multi-Container Demo
      - REDIS_HOST=redis
      - REDIS_PORT=6379
    depends_on:
      - redis
    networks:
      - app_net
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    container_name: redis_db
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis_data:/data
    networks:
      - app_net
    restart: unless-stopped

volumes:
  redis_data:

networks:
  app_net:
    driver: bridge
```

Step 3: Build and run the application

```
cd Assignment5_MultiContainer
docker compose up -d --build
docker compose ps
docker compose logs web
```

Step 4: Verify networking, volumes and environment variables

```
# Web app response (open http://localhost:8000 in a browser too)
curl http://localhost:8000

# Networks and volumes created by Compose
docker network ls
docker network inspect assignment5_multicontainer_app_net
docker volume ls

# Environment variables inside the web container
docker compose exec web env | findstr "APP_NAME REDIS"

# Query the database directly
docker compose exec redis redis-cli GET visits
```

Step 5: Test data persistence and clean up

```
docker compose down      # removes containers and network, keeps volume
docker compose up -d     # counter continues from previous value
docker compose down -v   # removes containers, network AND volume
```

Output

1. Building the web image and starting both containers with docker compose up (Compose creates the network, the volume and the two containers):

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker --version; docker compose version
Docker version 29.8.0, build 88096ef
Docker Compose version v5.5.1
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose up -d --build
```

```
...
#13 sending tarball 12.6s done
#13 DONE 13.8s

#15 resolving provenance for metadata file
#15 DONE 0.9s
Image assignment5_multicontainer-web Built
Network assignment5_multicontainer_app_net Creating
Volume assignment5_multicontainer_redis_data Creating
Volume assignment5_multicontainer_redis_data Creating
Network assignment5_multicontainer_app_net Creating
Volume assignment5_multicontainer_redis_data Created
Volume assignment5_multicontainer_redis_data Created
Network assignment5_multicontainer_app_net Created
Network assignment5_multicontainer_app_net Created
Container redis_db Creating
Container redis_db Created
Container flask_web Creating
Container flask_web Created
Container redis_db Starting
Container redis_db Started
Container flask_web Starting
Container flask_web Started
```

2. Both services running, with host port 8000 mapped to container port 5000. Each request increments the counter stored in Redis:

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose ps
```

NAME	SERVICE	STATUS	PORTS
flask_web	web	Up 5 seconds	0.0.0.0:8000->5000/tcp, [::]:8000->5000/tcp
redis_db	redis	Up 6 seconds	6379/tcp

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> curl.exe http://localhost:8000
```

```
<h1>DevOps Multi-Container Demo</h1><p>This page has been visited 1 times.</p>
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> curl.exe http://localhost:8000
```

```
<h1>DevOps Multi-Container Demo</h1><p>This page has been visited 2 times.</p>
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> curl.exe http://localhost:8000
```

```
<h1>DevOps Multi-Container Demo</h1><p>This page has been visited 3 times.</p>
```

3. Web application opened in the browser at <http://localhost:8000>:



4. Networking and volumes: the user-defined bridge network app_net connects both containers, and the named volume redis_data is created:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker network ls
NETWORK ID          NAME                                     DRIVER  SCOPE
235bfecc6998        assignment5_multicontainer_app_net      bridge  local
a83f5a5e2995        bridge                                 bridge  local
c5921791e252        host                                   host    local
77a5771a8743        none                                    null    local

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker network inspect assignment5_multicontainer_app_net
assignment5_multicontainer_app_net driver=bridge subnet=172.18.0.0/16
  container=flask_web ip=172.18.0.3/16
  container=redis_db ip=172.18.0.2/16

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker volume ls
DRIVER    VOLUME NAME
local     assignment5_multicontainer_redis_data
local     buildx_buildkit_default_state
```

5. Environment variables passed to the web container and the visit counter stored in the Redis database:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose exec web env | findstr "APP_NAME REDIS"
REDIS_PORT=6379
APP_NAME=DevOps Multi-Container Demo
REDIS_HOST=redis

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose exec redis redis-cli GET visits
4
```

Web service logs showing the requests handled by the Flask container:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose logs web --tail 6 --no-log-prefix
Press CTRL+C to quit
172.18.0.1 - - [27/Sep/2026 19:45:15] "GET / HTTP/1.1" 200 -
172.18.0.1 - - [27/Sep/2026 19:45:15] "GET / HTTP/1.1" 200 -
172.18.0.1 - - [27/Sep/2026 19:45:15] "GET / HTTP/1.1" 200 -
172.18.0.1 - - [27/Sep/2026 19:45:18] "GET / HTTP/1.1" 200 -
172.18.0.1 - - [27/Sep/2026 19:45:20] "GET /favicon.ico HTTP/1.1" 404 -
```

6. Data persistence: after docker compose down and up the containers are recreated, but the visit counter continues from the previous value because it is stored in the redis_data volume:

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose down
```

```
Container flask_web Stopping
Container flask_web Stopped
Container flask_web Removing
Container flask_web Removed
Container redis_db Stopping
Container redis_db Stopped
Container redis_db Removing
Container redis_db Removed
Network assignment5_multicontainer_app_net Removing
Network assignment5_multicontainer_app_net Removed
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose up -d
```

```
Network assignment5_multicontainer_app_net Creating
Network assignment5_multicontainer_app_net Creating
Network assignment5_multicontainer_app_net Created
Network assignment5_multicontainer_app_net Created
Container redis_db Creating
Container redis_db Created
Container flask_web Creating
Container flask_web Created
Container redis_db Starting
Container redis_db Started
Container flask_web Starting
Container flask_web Started
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> curl.exe http://localhost:8000
<h1>DevOps Multi-Container Demo</h1><p>This page has been visited 5 times.</p>
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose exec redis redis-cli
i GET visits
5
```

7. Final cleanup: docker compose down -v removes the containers, the network and the volume, leaving nothing running:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose down -v
Container flask_web Stopping
Container flask_web Stopped
Container flask_web Removing
Container flask_web Removed
Container redis_db Stopping
Container redis_db Stopped
Container redis_db Removing
Container redis_db Removed
Network assignment5_multicontainer_app_net Removing
Volume assignment5_multicontainer_redis_data Removing
Volume assignment5_multicontainer_redis_data Removed
Network assignment5_multicontainer_app_net Removed

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker compose ps -a
NAME          IMAGE          COMMAND          SERVICE    CREATED   STATUS    PORTS

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment5_MultiContainer> docker volume ls --filter name=assignment5
DRIVER      VOLUME NAME
```

Conclusion

In this assignment, I learned about building a multi-container application with Docker Compose. I created a Flask web service and a Redis database service and configured them in a single compose.yaml file using port mapping, environment variables, a named volume for persistent data and a user-defined bridge network, which lets the containers talk to each other by service name. I also saw how the whole stack can be built, started, inspected and removed with a few simple commands.

References

1. Docker Compose Documentation – <https://docs.docker.com/compose/>
2. Compose File Reference – <https://docs.docker.com/reference/compose-file/>
3. Docker Networking Overview – <https://docs.docker.com/engine/network/>
4. Docker Volumes – <https://docs.docker.com/engine/storage/volumes/>
5. Flask Documentation – <https://flask.palletsprojects.com/>
6. Redis Docker Official Image – https://hub.docker.com/_/redis

Submission Links

GitHub Repository Link:

https://github.com/tanushkapatil/DevOps/tree/main/Assignment5_MultiContainer

Hosted / Output Link: <http://localhost:8000> (local output – runs on the local machine via Docker Compose)